

No-IMU Gait Optimization for a 12-Servo Hexapod

Preliminary Results

E90 Hexapod Project

April 8, 2026

1 What Was Done

We built and ran a Python optimization pipeline that tunes the hexapod’s gait parameters using an ODE body model, without any IMU feedback. The workflow mirrors a standard MATLAB `ode45 + fmincon` approach:

1. Initialize the 25 gait parameters randomly (multiple restarts).
2. Simulate body dynamics with `scipy.integrate.solve_ivp`.
3. Minimize a weighted cost function with `scipy.optimize.minimize` (SLSQP, bounded).
4. Select the best result across all restarts.

Key deliverables produced:

- `hexapod_no_imu_optimization.py` – runnable optimizer
- `no_imu_optimization_summary.json` – machine-readable results
- `no_imu_optimization_timeseries.csv` – full body trajectory

2 Model Summary

Item	Detail
Servos	12 × SG90 (9 g, stall torque 0.176 N·m)
Total mass	0.458 kg (chassis 0.24 + battery/elec. 0.11 + servos 0.108)
Tripod groups	$\mathcal{A}=\{\text{FL, MR, RL}\}$, $\mathcal{B}=\{\text{FR, ML, RR}\}$, π -offset
Joint commands	sinusoidal: $q_l(t) = q_{l,0} + A_l \sin(\omega t + \delta_l + \phi_l)$
Body states	forward position, vertical position, roll, pitch
Contact model	smooth sigmoid ground-reaction heuristic

The 25 optimization variables are:

Variable set	Count	Description
Abduction rest angles	6	standing leg splay per leg
Flexion rest angles	6	standing leg height per leg
Abduction phase shifts	6	timing offset for forward/back swing
Flexion phase shifts	6	timing offset for lift/plant
Gait frequency	1	global stride rate
Total	25	

The cost function penalizes: frequency error from target, roll/pitch oscillation, vertical oscillation, low forward speed, servo overload, contact imbalance, and deviation from a symmetric baseline.

3 Results

Optimized parameters (leg order: FL, FR, ML, MR, RL, RR)

	FL	FR	ML	MR	RL	RR
Abd. rest ($^{\circ}$)	10.0	10.0	0.0	0.0	-10.0	-10.0
Flx. rest ($^{\circ}$)	32.0	32.0	34.1	34.0	32.0	32.0
Abd. shift ($^{\circ}$)	0.0	-0.2	-0.2	0.0	-0.2	-0.2
Flx. shift ($^{\circ}$)	-89.6	-89.8	-90.1	-89.6	-90.1	-90.1

$$f^* = 1.8008 \text{ Hz} \quad (\text{target was } 1.8 \text{ Hz})$$

Performance metrics

Metric	Value	Note
Gait frequency	1.8008 Hz	on target
Roll RMS	0.244 rad	moderate
Pitch RMS	0.021 rad	low
Peak servo usage	12.8%	well within SG90 limits
<i>Metrics below need model refinement:</i>		
Forward speed	1.13 m/s	over-estimated (model lacks friction detail)
Vertical RMS	6.21 m	not physical; contact model too soft
Body dom. freq.	3.66 Hz	artifact of reduced dynamics

4 Key Takeaways

1. **Optimization framework is working.** The Python `solve_ivp` + SLSQP pipeline runs end-to-end with multiple random restarts and produces repeatable parameter sets.
2. **Symmetric posture is preferred.** The optimizer converged to a front/mid/rear pattern that closely mirrors the current firmware constants.
3. **Abduction phase shifts are near zero.** This validates the existing tripod timing structure; the optimizer found no benefit in shifting the forward/back swing timing.
4. **Flexion shifts converge to -90° .** Lift timing is a quarter-cycle behind abduction, meaning the leg lifts when abduction velocity is highest – physically sensible for ground clearance.
5. **Servo loading is low.** Peak estimated torque stays at about 13% of the SG90 stall limit, leaving headroom for added payload or faster gaits.

5 Limitations

This is a reduced-order trend-finding model, not a validated simulator. The vertical displacement, forward speed, and body-resonance estimates are not yet physically meaningful because the contact/ground-reaction model is too simplified. These quantities should improve once the model is calibrated against measured robot data.

6 Next Steps

1. Improve the ground-contact formulation so vertical dynamics stay bounded.
2. Calibrate against measured stride frequency, step length, and body oscillation.
3. Add IMU roll/pitch feedback gains as additional optimization variables.
4. Compare no-IMU baseline versus IMU-stabilized gait.
5. Map optimized degree-space parameters back to Maestro pulse-space firmware constants.